AstraNotes — Requirements-to-UML Traceability Matrix

CSEN 296B-2 — Week 5.2 Lab | Instructor: Paddu Melanahalli

1. Traceability Matrix

Req ID	Requirement	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
FR-1	The system shall allow a user to create a text note with a title and body, and persist it to local storage.	**Note** class with title, content, createdAt, lastModified attributes; **NoteService.createNote()**; **NoteRepository.save()**. Object diagram shows *note_204* as a concrete instance.	**Create Note** use case present. **Activity diagram** models the full create-and-save flow including input validation, persistence, and version capture.	NoteService and NoteRepository components shown in application tier; notes.db in local storage tier.	**Fully Traced**	Core feature has structural, behavioral, and deployment support. No gap.
FR-2	The system shall allow a user to edit an existing note and persist the updated content to local storage.	**Note.updateContent()**; **NoteService.editNote()**; **NoteRepository.save()** handles both create and update paths.	**Edit Note** use case present. No dedicated activity diagram — edit-and-save is assumed to follow a similar persistence path as create, but this is not explicitly modeled.	Same persistence path as create: NoteService → NoteRepository → LocalStorage.	**Partially Traced**	Structurally supported but behaviorally implicit. The edit workflow should have its own activity diagram or share one with create.
FR-3	The system shall allow a user to delete a note while preserving its version history for possible restoration.	**NoteRepository.delete(id)**; **NoteRepository** aggregates **VersionHistory** with a hollow diamond, meaning version records survive note deletion (this design choice was made deliberately to support FR-6).	**Delete Note** use case present. No activity diagram models the delete workflow, the cascade behavior, or what happens to the search index entry after deletion.	Same persistence path; no deployment-level evidence of delete-specific behavior.	**Weakly Traced**	Use case exists, structural support exists, but the behavioral and side-effect story (version retention, index removal) is not visualized.
FR-4	The system shall allow a user to search notes by keyword across title and body, and display matching results or a clear no-results message.	**SearchIndex** class with tokens map and query() method; **NoteService.searchNotes(query)**; **SearchIndex.indexNote()** and **removeFromIndex()** for index maintenance.	**Search Notes** use case present. No activity diagram for the search workflow — empty-query and no-results edge cases are not modeled.	SearchIndex shown as a separate component in the application tier.	**Partially Traced**	Structure is solid; behavioral modeling is missing. Edge cases that the Week 3.1 requirement refinement called out (empty input, no matches) have no visible support.

Req ID	Requirement	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
FR-5	The system shall allow a user to mark a note as private, with content protected via an encryption mechanism appropriate to the technical path.	**SecureNote** extends **Note** with isLocked and encryptionKeyRef attributes and lock/unlock methods. **EncryptionService** provides encrypt/decrypt operations. **UserSession** tracks unlock state.	**Mark Note Private** use case present. **Unlock SecureNote** use case extends it. **Activity diagram** includes a privacy branch with a dedicated *Encrypt content via EncryptionService* step.	EncryptionService shown as application component with a dashed *<<reads key>>* dependency on the OS Keychain node, which lives outside the local storage tier so encryption keys are isolated from note data.	**Fully Traced**	Strongest area of the design. Privacy is supported across all three view types and the deployment posture matches the privacy claim.
NFR-1	The system shall be local-first: all note data shall persist to the user's device and the system shall not require a network connection to function.	**NoteRepository** and **LocalStorage** classes encapsulate device-local persistence. No network or remote-service classes exist in the design.	Not primarily behavioral. The activity diagram's persistence step ends at LocalStorage, consistent with the constraint.	**Deployment diagram** is the primary evidence: everything runs inside a single User Laptop / Desktop node. No network boundary, no external service nodes, OS Keychain on the same device.	**Fully Traced**	Deployment view is the right place for this constraint and the evidence is explicit.
NFR-2	The system shall handle invalid save, load, or delete operations gracefully, reporting clear errors instead of crashing.	Not directly represented as classes. Implicit in NoteService method contracts and NoteRepository return values, but no explicit error-handling type appears in the design.	**Activity diagram** shows a validation-error branch with a loop back to input for the create-note flow. No other flows model failure (storage failure, missing note, corrupt data, load failure).	No deployment-level fault tolerance shown.	**Weakly Traced**	Failure handling is modeled only for the create-note input validation case. Storage and retrieval failures are not represented anywhere.

2. Traceability Metrics Summary

Metric	Count
Total requirements reviewed	7
Fully Traced	3
Partially Traced	2
Weakly Traced	2
Not Traced	0
Major UML elements without a clear requirement reason to exist	1

Element without a clear requirement reason: VersionHistory appears as a structural element in the class diagram (aggregated by NoteRepository) and is supported by the deployment view, but the original Week 1.2 requirement baseline did not include a version-history or restore-previous-version requirement. The *Restore Previous Version* use case was added during the Week 4.2 design pass to justify the class, but without a corresponding entry in the requirement baseline, VersionHistory is currently more design-led than requirement-led. Either a version-history requirement (e.g., FR-6: "The system shall preserve previous note versions and allow the user to restore an earlier version") should be added to the baseline, or VersionHistory should be removed from the design to match the scope that the requirements actually justify.

3. Gap Analysis

The AstraNotes design package is structurally stronger than it is behaviorally complete. The class and object diagrams provide solid evidence for nearly every requirement, but the use case and activity diagrams cover only a subset of the workflows the requirements imply. Three of the seven reviewed requirements ended up Partially or Weakly Traced primarily because their behavioral support is missing or thin.

The biggest gap is behavioral coverage. The activity diagram models create-and-save in detail, but edit, delete, and search workflows have no dedicated activity diagrams. For edit (FR-2) the implicit assumption is that it follows the same persistence path as create, but that should be made explicit. For delete (FR-3) the cascade behavior with VersionHistory — which is a deliberate design choice supported by the aggregation relationship — is not visualized anywhere. For search (FR-4) the edge cases identified during Week 3.1 requirement refinement (empty input, no results) are not modeled.

Failure handling (NFR-2) is the weakest area. The activity diagram shows a validation-error loop for input on the create flow, but no other flow models what should happen when storage fails, when a note is missing, or when stored data cannot be parsed. For a system that promises graceful failure as a non-functional requirement, having only one happy-path deviation modeled is insufficient.

VersionHistory is the one over-designed element. It exists in the class diagram, the deployment view, and a use case, but no requirement in the original baseline asks for version history. This is not a fatal problem, but it should be resolved before implementation: either by promoting version restoration to a real requirement (which the design already supports) or by removing VersionHistory from the design to match the smaller justified scope.

Recommended actions before implementation: (1) add activity diagrams for edit, delete, and search, even if compact; (2) extend failure-handling representation beyond input validation, at minimum to cover storage write failures and missing-note retrievals; (3) reconcile VersionHistory with the requirement baseline, ideally by adding FR-6 for version restoration so the existing design earns its place. None of these are large changes, but addressing them now is cheaper than discovering the gaps during implementation.

4. How AI Helped Build This Matrix

I used GitHub Copilot Chat to draft the initial matrix structure and to suggest column wording, then applied my own judgment for the actual mappings, status labels, and gap analysis. The AI was useful for two specific things: normalizing requirement language so it stayed consistent with the Week 3.1 refined baseline, and producing a first-pass mapping between requirements and UML elements that I could then audit.

Where I overrode the AI's suggestions:

Status labels. The first AI pass leaned toward labeling more requirements as Fully Traced than I thought was honest. For FR-2, FR-3, and FR-4, the AI saw the use case existed and the structural support was present and called those Fully Traced. I downgraded them because the behavioral coverage is genuinely thin — there is no edit activity diagram, no delete activity diagram, and no search activity diagram. Marking them Fully Traced would have hidden the actual design gap.

The VersionHistory finding. The AI did not flag VersionHistory as an over-designed element on its own. I caught it by checking the other direction of traceability — for each major class, is there a requirement that justifies it? VersionHistory came up empty against the original Week 1.2 baseline, which is the kind of one-way traceability gap the rubric specifically asks for.

The NFR-2 assessment. The AI initially treated the validation-error loop in the create-note activity diagram as evidence of system-wide failure handling. I disagreed. One input-validation loop in one workflow does not satisfy a non-functional requirement that applies to save, load, and delete operations across the system. That's why NFR-2 ended up Weakly Traced rather than Partially Traced.

The matrix structure and consistent wording came from AI assistance, but the actual evaluation of each row required reading the diagrams against the requirements myself. AI was faster than me at producing a credible-looking matrix; it was slower at producing an honest one.